

If there is the exist **BinaryHeap**'s instance variable myheap which has 10 elements as the Binary Tree following. Write statements in main program to insert the instance variable myheap with input is 14 and write the outputs to array by using class **BinaryHeap** 's methods that we learned only. แบบที่ 1

Write the steps for the insert() function when insert element 14 into the instance variable myheap. In each round, consists of

- 1) what are the values of the variable hole, and
- 2) what the comparison under the if-else condition that is checked.

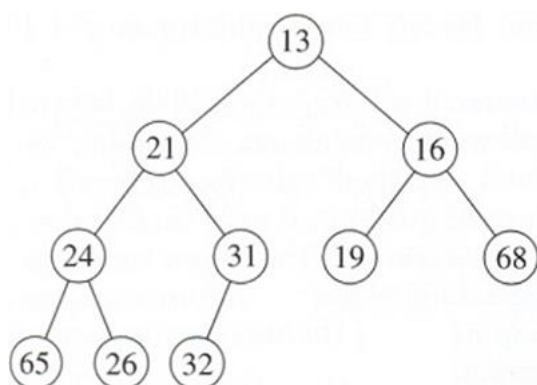
Each iteration must write 2 things.

Statements in main program:

myheap.insert(14)

print(myheap.array[1 : myheap.currentSize + 1])

[0]	ไม่มีข้อมูล
1	13
2	21
3	16
4	24
5	31
6	19
[7]	68
8	65
9	26
10	32
11	31
12	



hole = i (ตำแหน่งหลุม)

Left = 2 * hole

Right = 2 * hole + 1

ณ เวลานี้ ตัวแปรขอบเขต a มีสมาชิก 10 ตัว

[เฉลยการแทรก 14 และผลลัพธ์ของ Binary Heap]:

* สรุปขั้นตอนการแทรก 14 ใน Min-Heap (Percolate Up):

1. ขยายขนาด: currentSize เพิ่มจาก 10 เป็น 11 -> hole = 11
2. รอบ 1: hole=11, Parent 5 (31) -> 14 < 31 จริง -> เลื่อน 31 ลงช่อง 11, hole = 5
3. รอบ 2: hole=5, Parent 2 (21) -> 14 < 21 จริง -> เลื่อน 21 ลงช่อง 5, hole = 2
4. รอบ 3: hole=2, Parent 1 (13) -> 14 < 13 เท็จ! -> หยุดที่ตำแหน่งนี้
5. บรรจุ 14 ลงช่อง hole=2: array[2] = 14

* สถานะอาร์เรย์หลังแทรก: [None, 13, 14, 16, 24, 21, 19, 68, 65, 26, 32, 31]

```
def insert(self, x):
```

```
    if self.is_full():
```

```
        print("Heap is full")
```

```
    return
```

```
    self.currentSize += 1
```

```
    hole = self.currentSize
```

```
    # การหา parent: hole // 2
```

```
    while hole > 1 and x < self.array[hole // 2]:
```

```
        self.array[hole] = self.array[hole // 2]
```

```
        hole //= 2
```

```
    self.array[hole] = x
```

การทำงานในฟังก์ชัน insert

hole = 11

while loop รอบที่ 1 hole = 11, 11 > 1 จริง เปรียบเทียบ 14 < 31 (จริง) -> array[11]=31, hole=5

while loop รอบที่ 2 hole = 5, 5 > 1 จริง เปรียบเทียบ 14 < 21 (จริง) -> array[5]=21, hole=2

while loop รอบที่ 3 hole = 2, 2 > 1 จริง เปรียบเทียบ 14 < 13 (เท็จ) -> หลุดลูบ!

ออกจาก while loop ตัวแปร hole มีค่า เท่ากับเท่าไร = 2 -> บรรจุค่า: array[2] = 14

while loop ทำงานก็รอบ 2 รอบ ให้ดูว่า ประโยคคำสั่งที่อยู่ใน while loop ทำงานก็รอบ นั้นเอง

[คำอธิบายเจาะลึกและข้อควรระวังในห้องเรียน]:

* ข้อสังเกตและข้อควรระวังในการใส่โปรแกรม insert (Percolate Up):

1. สูตรหา Parent: $parent = hole // 2$ (ปัดเศษทิ้ง เช่น $11 // 2 = 5$, $5 // 2 = 2$)
2. การนับจำนวนรอบของ While Loop: ให้นับเฉพาะรอบที่ 'ประโยคคำสั่งภายในลูบทำงานสำเร็จ'
 - รอบที่ 1 (hole=11): คำสั่งทำงานสำเร็จ (ย้าย 31, hole เปลี่ยนเป็น 5)
 - รอบที่ 2 (hole=5): คำสั่งทำงานสำเร็จ (ย้าย 21, hole เปลี่ยนเป็น 2)
 - รอบที่ 3 (hole=2): เงื่อนไข $14 < 13$ เป็นเท็จ จึงไม่ทำคำสั่งภายในลูบและหลุดทันที-> ดังนั้น while loop จึงทำงานจริง 2 รอบ!
3. ค่า x=14 จะถูกนำมาใส่ลงใน array[hole] (ช่องที่ 2) นอก while loop เสมอ

```
def delete_min(self):
    if self.is_empty():
        print("Heap is empty")
        return
    min_item = self.array[1]
    self.array[1] = self.array[self.currentSize]
    self.currentSize -= 1
    self._percolate_down(1)
    return min_item
```

```
def _percolate_down(self, hole):
    temp = self.array[hole]
    while hole * 2 <= self.currentSize:
        child = hole * 2
        if child != self.currentSize and self.array[child + 1] < self.array[child]:
            child += 1
        if self.array[child] < temp:
            self.array[hole] = self.array[child]
        else:
            break
        hole = child
    self.array[hole] = temp
```

การทำงานในฟังก์ชัน **percolateDown**

hole = 1

tmp = 31 # ค่าจาก array[11] ตัวสุดท้าย (currentSize ลดเหลือ 10)

while loop รอบที่ 1 hole 1, 2 = 10 child 2 (Left Child) เปรียบเทียบ $16 < 14$ (เท็จ)

while loop รอบที่ 2 hole = 2 child = 4 เปรียบเทียบ $21 < 24$ (เท็จ) ~~ไม่ทำ~~ child++; -> child = 5 (จริง!)

while loop รอบที่ 3 hole = 5 child = 10 เปรียบเทียบ $32 < 31$ (เท็จ -> break!)

~~while loop รอบที่ 4 hole = , child = ? (ไม่ทำงาน / หลุดลูปตั้งแต่รอบ 3 ด้วย break)~~

ออกจาก while loop ตัวแปร child มีค่า เท่ากับเท่าไร = 10 (และ hole = 5 -> บรรจุ array[5] = 31)

while loop ทำงานกี่รอบ 2 รอบ

ตารางสถานะของ Array หลัง delete_min() เสร็จสมบูรณ์ (currentSize = 10):

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
-	14	21	16	24	31	19	68	65	26	32

[สรุปผลลัพธ์การทำงานของ percolateDown และข้อควรระวังข้อสอบปลายภาค]:

* สรุปขั้นตอนการทำงานของ percolateDown(1) ใน delete_min():

1. ดึงค่าน้อยที่สุดออก: min_item = array[1] (คือ 13)
 2. ดึงค่าตัวสุดท้ายมาเก็บที่ temp: temp = array[11] = 31, ขนาด currentSize ลดเหลือ 10
 3. รอบที่ 1: hole=1 -> เลือกลูกที่น้อยกว่าระหว่างลูกซ้าย (14) กับลูกขวา (16) -> ได้ลูกซ้าย (child=2)
 - เปรียบเทียบ: $14 < 31$ (จริง) -> เลื่อน 14 ขึ้นมาที่ array[1], ปรับหลุม hole = 2
 4. รอบที่ 2: hole=2 -> เลือกลูกที่น้อยกว่าระหว่างลูกซ้าย (24) กับลูกขวา (21) -> ได้ลูกขวา (child=5)
 - เปรียบเทียบ: $21 < 31$ (จริง) -> เลื่อน 21 ขึ้นมาที่ array[2], ปรับหลุม hole = 5
 5. รอบที่ 3: hole=5 -> ลูกซ้ายคือช่อง 10 (32), ไม่มีลูกขวา -> เปรียบเทียบ $32 < 31$ (เท็จ) -> break หลุดลูป!
 6. นำค่า temp มาใส่ในหลุม: array[hole] = temp -> array[5] = 31
- > ค่าเตือนข้อสอบ: เมื่อดู delete_min() ไม่มี parameter ห้ามใส่ค่าใดๆ เด็ดขาด (มิฉะนั้นได้ 0 คะแนน)